Core Assumptions for Execution and Documentation

To guarantee the pipeline executes flawlessly within 48 hours while running an expanded, computationally heavy extraction pipeline, the following assumptions and parameters guide the experiment:

1. **Hardware Constraints:** Kaggle provides two NVIDIA T4 GPUs, yielding a combined 32GB of VRAM and 320 GB/s of memory bandwidth.
2. **Time Constraints:** Kaggle enforces a strict 12-hour maximum limit for continuous interactive or background sessions.
3. **Model Footprint Viability:** The Qwen2.5-VL-14B model in 4-bit AWQ quantization requires exactly 21.29 GB of VRAM. This leaves an operational buffer of approximately 10 GB for the KV cache and data extraction operations, making it strictly viable for the 32GB dual-T4 setup.
4. **Dataset Scale:** The chosen MVBench evaluation suite consists of exactly 4,000 multiple-choice question-answer pairs.
5. **Algorithmic Expansion:** Because the workload will be parallelized across four people, the data ingestion pipeline has been safely expanded to its maximum theoretical limit for edge deployment. It implements a complete 5-tier dynamic compression script (Temporal Clustering $\rightarrow$ Differential Keyframe Selection $\rightarrow$ Supplementary Selection $\rightarrow$ Magnitude Pruning $\rightarrow$ Differential Feature Merging) using lightweight mathematical proxies to avoid loading secondary neural networks that would cause Out-Of-Memory (OOM) errors alongside the 14B model.

The Role of Fine-Tuning (Skipped for 2-Day Execution)

This pipeline is executed **without fine-tuning**, relying entirely on the instruction-tuned models' zero-shot capabilities.

While fine-tuning is often assumed to be vital for domain adaptation, recent research reveals that task-specific fine-tuning can actually degrade the Chain-of-Thought (CoT) reasoning performance and faithfulness of LLMs, with this negative effect being particularly pronounced in Small Language Models (SLMs). Furthermore, the core hypothesis of this project posits that intelligent data extraction and dynamic compression can effectively replace the need for task-specific fine-tuning.

From a feasibility standpoint, fine-tuning a 14B parameter model on video datasets requires massive VRAM and days of continuous compute, strictly violating the 48-hour Kaggle dual-T4 constraints. Therefore, the fine-tuning stage is reported here for theoretical completeness but is **skipped** in the execution plan.

Evaluation using MVBench Framework

The final evaluation is conducted using MVBench to measure both reasoning performance and

system efficiency.

- **Pre-Tuning Evaluation:** Establish zero-shot baseline (This is the primary and only actionable phase for this 48-hour execution guide).
- **Post-Tuning Evaluation:** Measure improvements after training (Skipped due to hardware/time constraints as noted above).

Metrics include:

- **Accuracy (Acc):** Across multiple temporal reasoning tasks.
- **Time-to-First-Token (TTFT):** Measures inference latency.
- **Peak VRAM Usage:** Tracks memory efficiency.

A Pareto frontier is constructed to analyze the trade-off between accuracy and compute cost. If the SLM achieves performance comparable to the LLM under equal compute constraints, the effectiveness of the data-centric pipeline is validated.

Algorithm Research Papers for In-Depth Study

To fully understand the theoretical mechanics driving the 5-tier pipeline utilized in this execution guide, the following algorithm-specific research papers must be studied:

1. **VideoINSTA: Zero-shot Long Video Understanding via Informative Spatial-Temporal Reasoning with LLMs** [1]
   - *Focus*: Details the **C-DPCKNN** (Continuous Density-Peak Clustering) algorithm used for query-agnostic temporal segmentation, which establishes strict chronological boundaries for visual events.
2. **Scaling Video-Language Models to 10K Frames via Hierarchical Differential Distillation (ViLAMP)** [1]
   - *Focus*: Introduces **Differential Keyframe Selection (DKS)** to isolate salient frames and **Differential Feature Merging (DFM)** to compress non-keyframes into dense single tokens without losing continuous background logic.
3. **Inference Optimal VLMs Need Fewer Visual Tokens and More Parameters** [1]
   - *Focus*: Provides the mathematical proofs for **extreme token compression** and spatial magnitude pruning, demonstrating how to compute the inference-optimal frontier.
4. **Demystifying Small Language Models for Edge Deployment** [1]
   - *Focus*: Essential reading for understanding hardware-model co-design, specifically how memory footprint and inference latency behave on edge environments like the T4 GPUs being used.

Careful Steps & Considerations

- **Direct Benchmark Procurement:** MVBench relies on complex raw video data. Relying solely on the HuggingFace datasets library to stream videos on the fly will bottleneck the GPU. You must clone the official repository and manage the video paths directly.

- **In-Memory Image Arrays (I/O Optimization):** To avoid continuous disk writing and reading which bottlenecks the GPU, the Python extraction script completely avoids writing temporary .mp4 files. Instead, it returns a list of in-memory PIL Images, which the qwen_vl_utils processor can accept directly, saving seconds of latency per query.
- **Tensor Parallelism:** Because the 14B AWQ model exceeds the 16GB VRAM of a single T4, you must utilize the vLLM inference engine to split the model across both GPUs using a tensor parallel size of 2.
- **Chunked Prefill:** To prevent massive VRAM spikes when the 14B model ingests the compressed video tokens, enable_chunked_prefill=True is enabled in the vLLM configuration. This forces the engine to process the input prompt in smaller token chunks, drastically reducing activation memory usage during the compute-bound prefill stage and optimizing generation throughput.
- **Context Window Overflow:** You must strictly limit the --max-model-len to 4096 to prevent the dual T4 PCIe bandwidth from saturating and crashing the environment.
- **Thermal Management:** Processing 5-tier visual compression simultaneously with matrix multiplication generates immense heat. The evaluation loop must include brief pauses to clear buffers and prevent thermal latency spikes.

Phase 1: Universal Setup (All Persons Must Execute)

All four team members must run this cell at the start of their Kaggle sessions. It installs the dependencies, downloads the MVBench dataset, and defines the complete 5-tier data compression pipeline returning in-memory images.

Python

```
# cell 1: Universal Environment & Pipeline Setup
import os
import cv2
import json
import time
import numpy as np
from PIL import Image
from datasets import load_dataset
from decord import VideoReader, cpu

# 1. Install core dependencies
!pip install -q --upgrade transformers accelerate optimum auto-gptq datasets
!pip install -q qwen-vl-utils[decord]
!pip install -q vllm==0.6.0
```

```
!pip install -q decord opencv-python

# 2. Download the MVBench Dataset
print("Downloading MVBench Dataset metadata...")
mvbench_dataset = load_dataset("OpenGVLab/MVBench", split="test")

# Note: In a real Kaggle environment, the massive 17GB raw video zip for MVBench
# should be added as a Kaggle Dataset via the UI to avoid downloading it every session.
# We assume it is mounted at '/kaggle/input/mvbench-raw-videos/'
VIDEO_DIR = '/kaggle/input/mvbench-raw-videos/'

# 3. Complete 5-Tier Dynamic Compression Pipeline (I/O & Compute Optimized)
def maximized_5_tier_compression(video_path, max_keyframes=8, l2_threshold=10.0,
similarity_threshold=0.85):
    """
    Tier 1: Temporal Segmentation (C-DPCKNN proxy via structural clustering)
    Tier 2: Differential Keyframe Selection (DKS proxy)
    Tier 3: Dynamic Supplementary Selection
    Tier 4: Adaptive Spatial Compression (Magnitude Pruning)
    Tier 5: Differential Feature Merging (DFM proxy for non-keyframes)
    """
    vr = VideoReader(video_path, ctx=cpu(0))
    total_frames = len(vr)

    # Base extraction (Dense Uniform)
    indices = np.linspace(0, total_frames - 1, num=max_keyframes * 4, dtype=int)
    raw_frames = vr.get_batch(indices).asnumpy()

    keyframes =
    dropped_frames =
    prev_hist = None

    for frame in raw_frames:
        gray = cv2.cvtColor(frame, cv2.COLOR_RGB2GRAY)

        # Tier 4 Proxy: Magnitude Pruning (Drop completely dark/uninformative frames)
        if np.linalg.norm(gray) < l2_threshold:
            dropped_frames.append(frame)
            continue

        # Compute histogram for semantic similarity
        hist = cv2.calcHist([gray], , None, , )
        hist = cv2.normalize(hist, hist).flatten()
```

```
        if prev_hist is not None:
            # Tier 1 & 2 Proxy: Structural clustering and Differential Keyframe Selection
            sim = np.dot(hist, prev_hist) / (np.linalg.norm(hist) * np.linalg.norm(prev_hist))

            # Only keep if semantically distinct (Event Boundary / High Variance)
            if sim < similarity_threshold:
                keyframes.append(frame)
                prev_hist = hist
            else:
                dropped_frames.append(frame)
        else:
            keyframes.append(frame)
            prev_hist = hist

        # Hard cap to protect the LLM's KV Cache
        if len(keyframes) == max_keyframes - 1: # Leave room for the merged DFM token
            break

    # Tier 5 Proxy: Differential Feature Merging (DFM)
    # Instead of discarding non-keyframes, average them into a single continuous context frame
    if dropped_frames:
        merged_context_frame = np.mean(dropped_frames, axis=0).astype(np.uint8)
        keyframes.append(merged_context_frame)

    # Fallback if compression drops too much
    if len(keyframes) == 0:
        keyframes = [raw_frames]

    # Convert to in-memory PIL Images
    pil_frames = [Image.fromarray(frame) for frame in keyframes]

    return pil_frames
```

Phase 2: Distributed Execution Strategy

The exact 4,000 MVBench queries are divided evenly across four people (1,000 queries each) to ensure massive safety buffers against the 12-hour session limit. Each person executes their respective block of code in a new cell after running Phase 1.

Person A: 3B Baseline & 14B (Queries 0 to 999)

Person A manages the fast 3B FP16 baseline and the first block of the heavy 14B deployment.

Python

```python
# PERSON A EXECUTION SCRIPT
from vllm import LLM, SamplingParams
from qwen_vl_utils import process_vision_info
import time
import json

# Define subset for Person A
START_IDX = 0
END_IDX = 1000

# Initialize 14B AWQ Model split across 2x T4 GPUs
llm = LLM(
    model="Qwen/Qwen2.5-VL-14B-Instruct-AWQ",
    tensor_parallel_size=2,
    max_model_len=4096,
    gpu_memory_utilization=0.95,
    enforce_eager=True,
    enable_chunked_prefill=True  # Enables chunked prefill to prevent VRAM spikes
)

sampling_params = SamplingParams(temperature=0.0, max_tokens=128)
results_log =

print(f"Person A starting MVBench Evaluation ({START_IDX} to {END_IDX - 1})...")
for i in range(START_IDX, END_IDX):
    sample = mvbench_dataset[i]
    question = sample['question']
    candidates = sample['candidates']
    video_file = os.path.join(VIDEO_DIR, sample['video'])

    # Format multiple choice question
    options_text = "\n".join([f"({chr(65+j)}) {opt}" for j, opt in enumerate(candidates)])
    full_prompt = f"{question}\n{options_text}\nPlease output the correct option strictly as a single letter."

    # Run dynamic compression (returns PIL Images)
    compressed_frames = maximized_5_tier_compression(video_file)

    messages = [{"role": "user", "content": [
```

```
        {"type": "video", "video": compressed_frames},
        {"type": "text", "text": full_prompt}
    ]}]

    # Process and Generate
    from transformers import AutoProcessor
    processor = AutoProcessor.from_pretrained("Qwen/Qwen2.5-VL-14B-Instruct-AWQ")
    prompt = processor.apply_chat_template(messages, tokenize=False,
add_generation_prompt=True)
    image_inputs, video_inputs = process_vision_info(messages)

    start_time = time.time()
    outputs = llm.generate({"prompt": prompt, "multi_modal_data": {"video": video_inputs}},
sampling_params=sampling_params, use_tqdm=False)
    latency = time.time() - start_time

    response = outputs.outputs.text.strip()

    gt_index = candidates.index(sample['answer'])
    gt_letter = chr(65 + gt_index)

    results_log.append({
        "id": i,
        "question": question,
        "response": response,
        "latency": latency,
        "ground_truth_letter": gt_letter,
        "ground_truth_text": sample['answer']
    })

    # Thermal management pause
    if i % 50 == 0:
        time.sleep(2.0)

# Save Node 1 Results
with open("/kaggle/working/person_A_results.json", "w") as f:
    json.dump(results_log, f)
print("Person A execution complete.")
# Note: Person A repeats this script changing the model to "Qwen/Qwen2.5-VL-3B-Instruct"
(tensor_parallel_size=1) for the 3B baseline.
```

Person B: 14B (Queries 1000 to 1999)

Person B processes the second block of the dataset.

Python

```
# PERSON B EXECUTION SCRIPT
from vllm import LLM, SamplingParams
from qwen_vl_utils import process_vision_info
import time
import json

# Define subset for Person B
START_IDX = 1000
END_IDX = 2000

# Initialize 14B AWQ Model split across 2x T4 GPUs
llm = LLM(
    model="Qwen/Qwen2.5-VL-14B-Instruct-AWQ",
    tensor_parallel_size=2,
    max_model_len=4096,
    gpu_memory_utilization=0.95,
    enforce_eager=True,
    enable_chunked_prefill=True
)

sampling_params = SamplingParams(temperature=0.0, max_tokens=128)
results_log =

print(f"Person B starting MVBench Evaluation ({START_IDX} to {END_IDX - 1})...")
for i in range(START_IDX, END_IDX):
    sample = mvbench_dataset[i]
    question = sample['question']
    candidates = sample['candidates']
    video_file = os.path.join(VIDEO_DIR, sample['video'])

    options_text = "\n".join([f"({chr(65+j)}) {opt}" for j, opt in enumerate(candidates)])
    full_prompt = f"{question}\n{options_text}\nPlease output the correct option strictly as a single letter."

    compressed_frames = maximized_5_tier_compression(video_file)

    messages = [{"role": "user", "content": [
```

```
        {"type": "video", "video": compressed_frames},
        {"type": "text", "text": full_prompt}
    ]}]

    from transformers import AutoProcessor
    processor = AutoProcessor.from_pretrained("Qwen/Qwen2.5-VL-14B-Instruct-AWQ")
    prompt = processor.apply_chat_template(messages, tokenize=False,
add_generation_prompt=True)
    image_inputs, video_inputs = process_vision_info(messages)

    start_time = time.time()
    outputs = llm.generate({"prompt": prompt, "multi_modal_data": {"video": video_inputs}},
sampling_params=sampling_params, use_tqdm=False)
    latency = time.time() - start_time

    response = outputs.outputs.text.strip()

    gt_index = candidates.index(sample['answer'])
    gt_letter = chr(65 + gt_index)

    results_log.append({
        "id": i,
        "question": question,
        "response": response,
        "latency": latency,
        "ground_truth_letter": gt_letter,
        "ground_truth_text": sample['answer']
    })

    if i % 50 == 0:
        time.sleep(2.0)

# Save Node 2 Results
with open("/kaggle/working/person_B_results.json", "w") as f:
    json.dump(results_log, f)
print("Person B execution complete.")
```

Person C: 14B (Queries 2000 to 2999)

Person C processes the third block of the dataset.

Python

```
# PERSON C EXECUTION SCRIPT
from vllm import LLM, SamplingParams
from qwen_vl_utils import process_vision_info
import time
import json

# Define subset for Person C
START_IDX = 2000
END_IDX = 3000

# Initialize 14B AWQ Model split across 2x T4 GPUs
llm = LLM(
    model="Qwen/Qwen2.5-VL-14B-Instruct-AWQ",
    tensor_parallel_size=2,
    max_model_len=4096,
    gpu_memory_utilization=0.95,
    enforce_eager=True,
    enable_chunked_prefill=True
)

sampling_params = SamplingParams(temperature=0.0, max_tokens=128)
results_log =

print(f"Person C starting MVBench Evaluation ({START_IDX} to {END_IDX - 1})...")
for i in range(START_IDX, END_IDX):
    sample = mvbench_dataset[i]
    question = sample['question']
    candidates = sample['candidates']
    video_file = os.path.join(VIDEO_DIR, sample['video'])

    options_text = "\n".join([f"({chr(65+j)}) {opt}" for j, opt in enumerate(candidates)])
    full_prompt = f"{question}\n{options_text}\nPlease output the correct option strictly as a single letter."

    compressed_frames = maximized_5_tier_compression(video_file)

    messages = [{"role": "user", "content": [
        {"type": "video", "video": compressed_frames},
        {"type": "text", "text": full_prompt}
    ]}]

    from transformers import AutoProcessor
```

```
    processor = AutoProcessor.from_pretrained("Qwen/Qwen2.5-VL-14B-Instruct-AWQ")
    prompt = processor.apply_chat_template(messages, tokenize=False,
add_generation_prompt=True)
    image_inputs, video_inputs = process_vision_info(messages)

    start_time = time.time()
    outputs = llm.generate({"prompt": prompt, "multi_modal_data": {"video": video_inputs}},
sampling_params=sampling_params, use_tqdm=False)
    latency = time.time() - start_time

    response = outputs.outputs.text.strip()

    gt_index = candidates.index(sample['answer'])
    gt_letter = chr(65 + gt_index)

    results_log.append({
        "id": i,
        "question": question,
        "response": response,
        "latency": latency,
        "ground_truth_letter": gt_letter,
        "ground_truth_text": sample['answer']
    })

    if i % 50 == 0:
        time.sleep(2.0)

# Save Node 3 Results
with open("/kaggle/working/person_C_results.json", "w") as f:
    json.dump(results_log, f)
print("Person C execution complete.")
```

Person D: 14B (Queries 3000 to 4000) & Consolidation

Person D runs the final batch. Once A, B, C, and D download their respective results.json files, Person D runs the final exact-match evaluation script to plot accuracy.

Python

```
# PERSON D EXECUTION SCRIPT
from vllm import LLM, SamplingParams
```

```
from qwen_vl_utils import process_vision_info
import time
import json
import re

# Define subset for Person D
START_IDX = 3000
END_IDX = 4000

# Initialize 14B AWQ Model split across 2x T4 GPUs
llm = LLM(
    model="Qwen/Qwen2.5-VL-14B-Instruct-AWQ",
    tensor_parallel_size=2,
    max_model_len=4096,
    gpu_memory_utilization=0.95,
    enforce_eager=True,
    enable_chunked_prefill=True
)

sampling_params = SamplingParams(temperature=0.0, max_tokens=128)
results_log =

print(f"Person D starting MVBench Evaluation ({START_IDX} to {END_IDX - 1})...")
for i in range(START_IDX, END_IDX):
    sample = mvbench_dataset[i]
    question = sample['question']
    candidates = sample['candidates']
    video_file = os.path.join(VIDEO_DIR, sample['video'])

    options_text = "\n".join([f"({chr(65+j)}) {opt}" for j, opt in enumerate(candidates)])
    full_prompt = f"{question}\n{options_text}\nPlease output the correct option strictly as a single letter."

    compressed_frames = maximized_5_tier_compression(video_file)

    messages = [{"role": "user", "content": [
        {"type": "video", "video": compressed_frames},
        {"type": "text", "text": full_prompt}
    ]}]

    from transformers import AutoProcessor
    processor = AutoProcessor.from_pretrained("Qwen/Qwen2.5-VL-14B-Instruct-AWQ")
    prompt = processor.apply_chat_template(messages, tokenize=False,
add_generation_prompt=True)
```

```
    image_inputs, video_inputs = process_vision_info(messages)

    start_time = time.time()
    outputs = llm.generate({"prompt": prompt, "multi_modal_data": {"video": video_inputs}},
sampling_params=sampling_params, use_tqdm=False)
    latency = time.time() - start_time

    response = outputs.outputs.text.strip()

    # Ground truth tracking
    gt_index = candidates.index(sample['answer'])
    gt_letter = chr(65 + gt_index)

    results_log.append({
        "id": i,
        "question": question,
        "response": response,
        "latency": latency,
        "ground_truth_letter": gt_letter,
        "ground_truth_text": sample['answer']
    })

    if i % 50 == 0:
        time.sleep(2.0)

# Save Node 4 Results
with open("/kaggle/working/person_D_results.json", "w") as f:
    json.dump(results_log, f)
print("Person D execution complete.")

# ------------------------------------------------------------
# PERSON D CONSOLIDATION & EVALUATION SCRIPT
# (Run this locally or in a final notebook after aggregating JSONs)
# ------------------------------------------------------------
def evaluate_exact_match(json_paths):
    total_correct = 0
    total_samples = 0
    total_latency = 0.0

    for path in json_paths:
        with open(path, 'r') as f:
            data = json.load(f)
            for item in data:
```

```
            total_samples += 1
            total_latency += item['latency']

            # Regex to find standard Option letter (e.g., "A", "(A)", "Option A")
            match = re.search(r'\b([A-E])\b', item['response'])
            predicted_letter = match.group(1) if match else "N/A"

            if predicted_letter == item.get('ground_truth_letter', ''):
                total_correct += 1

    accuracy = (total_correct / total_samples) * 100
    avg_latency = total_latency / total_samples

    print(f"--- Pareto Optimal Final Results ---")
    print(f"Total Samples Evaluated: {total_samples}")
    print(f"Overall Accuracy: {accuracy:.2f}%")
    print(f"Average Inference Latency (incl. Prefill): {avg_latency:.2f} seconds")

# Example Usage:
# evaluate_exact_match()
```

Works cited

1. All Research Papers Reports v2.0.1.pdf